



Tribhuvan University

Faculty of Humanities and Social Science

GROCERY DELIVERY SYSTEM

“HARIYO BAZZAR”

A PROJECT REPORT

Submitted to

Department of Computer Application

National College

In partial fulfillment of the requirement for the Bachelor's in computer application

Submitted by

Manish Raya Bhat

TU Reg no: 6-2-366-41-2021

&

Sujal K.C

TU Reg no: 6-2-366-52-2021

18 Shrawan 2082

Under the Supervision of

Buddhi Budhathoki



Tribhuvan University

Faculty of Humanities and Social Science

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Manish Raya Bhat & Sujal K.C entitled “Grocery Delivery System” in partial fulfillment of the Requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

SIGNATURE

Buddhi Budhathoki

Supervisor

National College Lainchaur, Kathmandu



Tribhuvan University

Faculty of Humanities and Social Sciences

LETTER OF APPROVAL

This is to certify that this project prepared by **Manish Raya Bhat & Sujal K.C** entitled “Grocery Delivery System” in partial fulfillment of the requirements for the degree of Bachelor’s in computer application has been evaluated. In our opinion, it is satisfactory in the scope and quality as a project for the required degree.

Buddhi Budhathoki Supervisor National College Lainchaur, Kathmandu	Navaraj Heka Computer Science National College Lainchaur, Kathmandu
Internal Examiner	External Examiner

ABSTRACT

In today's rapidly evolving e-commerce landscape, providing an intuitive, secure, and scalable platform for both consumers and sellers is paramount. HARIYOBAZZAR is an e-commerce web application designed to cater to the needs of both online shoppers and merchants. It offers a range of features, including user and seller registration, product browsing, product management, secure payment processing, and order management. With its sleek and responsive user interface, HARIYOBAZZAR is built to enhance the online shopping experience while providing sellers with the tools needed to effectively run their businesses.

The system empowers users to easily search and purchase products, manage their orders and addresses, and submit product reviews. For sellers, the platform provides a dedicated dashboard to manage product listings, process orders, and oversee customer interactions. Through robust authentication mechanisms using JWT and cookie-based sessions, the application ensures secure access for both users and sellers.

Built on the (MongoDB, Express.js, React, Node.js), HARIYOBAZZAR leverages modern technologies such as React for dynamic front-end interactions, Tailwind CSS for responsive design, and Axios for seamless API communication. Image handling is efficiently managed via Cloudinary, and secure payments are processed using Stripe in test mode. The platform also integrates advanced features like real-time toast notifications, secure API routing, and middleware-based access control, ensuring a high level of usability and security.

This e-commerce solution is designed to be scalable and modular, supporting future extensions and integrations to meet the evolving demands of both customers and sellers.

Keywords: E-commerce, Online Shopping, Seller Management, Secure Payments, Product Management, Scalable Platform, JWT Authentication, Stripe Payment Integration.

ACKNOWLEDGEMENT

We extend our sincere gratitude to National College for their invaluable support and provision of a platform for our project. Additionally, we wish to express our deep appreciation to the individuals who generously offered their insights, suggestions, and guidance throughout its development. Their contributions have been instrumental in our success.

We wish to express our sincere gratitude to our supervisor Mr. Buddhi Budhathoki for entrusting us with the opportunity to undertake this remarkable project on the topic “Grocery Management System” His guidance has been invaluable in enhancing our knowledge and capabilities. We are also thankful for his consistent feedback and encouragement throughout the project. Without his guidance and support, the project would not have been accomplished. Additionally, we are grateful for the wealth of knowledge and insight gained during the courses of this project under his mentorship.

Furthermore, we are grateful for the unwavering support and guidance extended to us by our parents, friends, and seniors. Their assistance throughout the project has been invaluable, offering innovative ideas and timely help whenever needed. So, we would like to thank them for their contributions in making this project absolutely wonderful and unique endeavor.

Finally, we express our gratitude to all those who have constantly provided encouragement and assistance, helping us navigate and overcome the challenges encountered during the completion of this project. Your unwavering support has been invaluable, and we deeply appreciate your contributions.

Manish Raya Bhat & Sujal K.C

July 2025

Table of Contents

ABSTRACT	iii
ACKNOWLEDGEMENT.....	iv
List Of Figures.....	vii
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction	1
1.2 Problem Statement.....	3
1.3 Objectives.....	3
1.4 Scope and Limitations	3
1.5 Development Methodology.....	6
1.6 Report Organization	7
CHAPTER 2: LITERATURE REVIEW.....	8
2.1 Background Study	8
2.2 Literature Review	8
Chapter 3: System Analysis and Design.....	10
3.1 System Analysis	10
3.1.1 Requirement Analysis	10
3.1.2 Feasibility Study.....	14
3.1.3 Data Modeling (ER-Diagram)	15
3.1.4 Process Modeling DFD	16
3.2 System Design.....	19
3.2.1 Architectural Design	19
Chapter 4: IMPLEMENTATION AND TESTING	20
4.1 Implementation	20
4.1.1 Tools Used	20
4.1.2 Implementation details of modules.....	20
4.2 Testing	22
4.2.1 Test Cases for Unit Testing.....	22
Chapter 5: RESULT CONCLUSION AND REMAINING WORK.....	25
5.1 Result.....	25

5.2 Conclusion	25
5.3 Remaining work.....	25
Reference	27

List Of Figures

Figure 1: Waterfall Model for Grocery Delivery System.....	6
Figure 2: Gantt Chart for Grocery Delivery System	6
Figure 3: Use Case Diagram Of Grocery Delivery System	12
Figure 4: ER-Diagram of Grocery Delivery System.....	15
Figure 5: Data Flow Diagram of Grocery Delivery System.....	16
Figure 6: Level one Context Diagram of Grocery Delivery System.....	17
Figure 7: Level two Context Diagram of Grocery Delivery System.....	18
Figure 8: Architectural Design of Grocery Delivery System	19

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid advancement of information and communication technology in recent decades has revolutionized the way businesses operate. With the widespread availability of high-speed internet, smartphones, and digital infrastructure, traditional commerce is steadily transitioning to electronic commerce, commonly known as e-commerce. E-commerce platforms have become integral to modern business operations, providing enhanced convenience, broader reach, and improved efficiency to both consumers and vendors. Customers no longer need to visit physical stores to purchase products; instead, they can order items from the comfort of their homes with just a few clicks. Similarly, sellers now have access to a vast digital marketplace where they can promote and sell their goods to a wider audience without the constraints of geographical boundaries.

In this context, the demand for localized, user-friendly, and robust e-commerce platforms has increased significantly. Many small and mid-scale vendors in developing regions still face challenges when attempting to establish an online presence due to a lack of technical skills, infrastructure, or resources. Most global e-commerce platforms do not cater to the specific needs of local sellers and buyers, especially in terms of user experience, cultural context, payment flexibility, and logistics. Recognizing this gap, **HARIYOBABAZAR** was conceptualized and developed as a e-commerce solution aimed at empowering local sellers and facilitating digital shopping for end-users.

HARIYOBABAZAR is a fully functional web-based application developed using the MERN (MongoDB, Express.js, React, Node.js) stack. This modern tech stack is well-suited for scalable, high-performance, and real-time web applications. The project integrates frontend and backend components in a modular structure, ensuring efficient communication between the user interface and server-side logic. On the frontend, the application utilizes React for building dynamic interfaces, Vite for fast bundling and development, and Tailwind CSS for responsive and aesthetic styling. On the backend, Express.js and Node.js handle API routing, middleware, and business logic, while MongoDB manages data persistence with flexibility and scalability.

The application serves two primary roles: users and sellers. End-users can register or log in, browse products categorized into different sections, view product details, manage a

shopping cart, place orders, and track their purchases. Additionally, users can save multiple shipping addresses, submit product reviews, and contact support. Sellers, on the other hand, are provided with a dedicated dashboard where they can register/login, add or edit product listings (with multiple images using Cloudinary integration), monitor orders, manage stock availability, and interact with customer messages.

Security and usability are key priorities in this system. User and seller authentication is handled via JWT (JSON Web Tokens) and secured using encrypted cookies, ensuring safe login sessions. Payment functionality is implemented using Stripe in test mode, allowing simulation of secure transaction flows. The application provides instant feedback to users through toast notifications and includes loading/error boundaries to enhance the overall user experience.

Moreover, HARIYOBAZZAR features a clean and modern UI that adapts seamlessly to different screen sizes and devices, ensuring accessibility and usability for a wider audience. The entire codebase follows component-based and modular development practices, enabling easy maintenance, scalability, and future upgrades.

In summary, HARIYOBAZZAR is a technically sound, user-friendly, and business-oriented web application that not only reflects the practical knowledge gained over the BCA course but also addresses a real-world problem. It showcases how modern development can be utilized to create powerful, interactive, and socially impactful digital platforms that serve both businesses and customers in a meaningful way.

1.2 Problem Statement

In many developing regions, small and mid-scale sellers lack access to modern e-commerce platforms. They face challenges in managing product listings, customer orders, and payment systems efficiently. On the user end, there is often a lack of reliable, localized platforms that support intuitive browsing, seamless transactions, and responsive design.

The problem, therefore, lies in the absence of a robust, scalable, and easy-to-use digital solution that caters to both users and sellers—especially in a local marketplace context.

1.3 Objectives

The objective of this project is:

- To deliver fast and fresh Grocery Items.
- To allow customer to order grocery from anywhere.
- To provide real-time update on orders and delivery tracking.

1.4 Scope and Limitations

Scope

The scope of this project is defined by the actual features and functionalities developed in HARIYOBAZZAR, focused on providing an efficient e-commerce experience for both customers and sellers.

User and Seller Role Management:

The system supports two separate roles—users and sellers—each with dedicated login and dashboards, ensuring role-based access and control over system functionalities.

Product Management:

Sellers can add, update, and delete products, including uploading multiple images using Cloudinary. They can also manage stock availability and set offer prices.

Order Management:

Customers can place orders through a simple and secure checkout process. Sellers can view, process, and manage customer orders from their dashboard.

Cart and Address Management:

Customers can add products to their cart, manage cart items, and save multiple addresses for order delivery.

Authentication and Authorization:

Secure authentication is implemented using JWT and cookies for both users and sellers, protecting sensitive data and ensuring session-based access.

Image and Asset Handling:

Product images and assets are managed via Cloudinary, allowing smooth and reliable media uploads without local server load.

Payment Integration:

Stripe is integrated in test mode, allowing customers to simulate digital payments during checkout.

UI and Responsiveness:

The frontend is built using React and Tailwind CSS with a modern, responsive design for both desktop and mobile browsers.

Notifications and Feedback:

Real-time feedback is provided to users and sellers using React Hot Toast, enhancing user experience during actions like login, cart updates, and order placemen

Limitation

While HARIYOBAZZAR implements essential e-commerce functionalities, the current version of the system has some limitations:

Stripe in Test Mode Only:

Payment integration is restricted to test mode only. Real-time transactions are not enabled, limiting real-world usability for customers.

No Admin Role:

The application lacks a centralized admin panel for platform-wide control such as monitoring users, approving sellers, or managing system-wide configurations.

Basic Search and Filter Options:

The system allows product browsing by categories only. There is no advanced filtering (by price, ratings, brand, etc.) or sorting features implemented.

No Real-Time Order or Delivery Tracking:

Order status updates are manual. There is no integration with delivery APIs or automated shipment tracking.

Limited Product Categorization:

Product categories are basic and not dynamic or configurable through the dashboard. New categories must be added manually via backend/database.

No Email or SMS Notification System:

Users and sellers do not receive order confirmation, dispatch alerts, or promotional notifications via email or SMS.

No Mobile App:

The system is fully web-based. There is no Android or iOS application for mobile users.

1.5 Development Methodology

I used the waterfall model for the development of this project because the waterfall model has well-defined phases such as requirements, design, implementation, testing, and maintenance. The waterfall model also provides a structured and predictable progress path and since the waterfall model is well suited for projects with clear and well-defined requirements from the beginning and in Grocery Delivery System functionality are known like tracking stock levels, managing orders, etc. hence waterfall model will be effective.

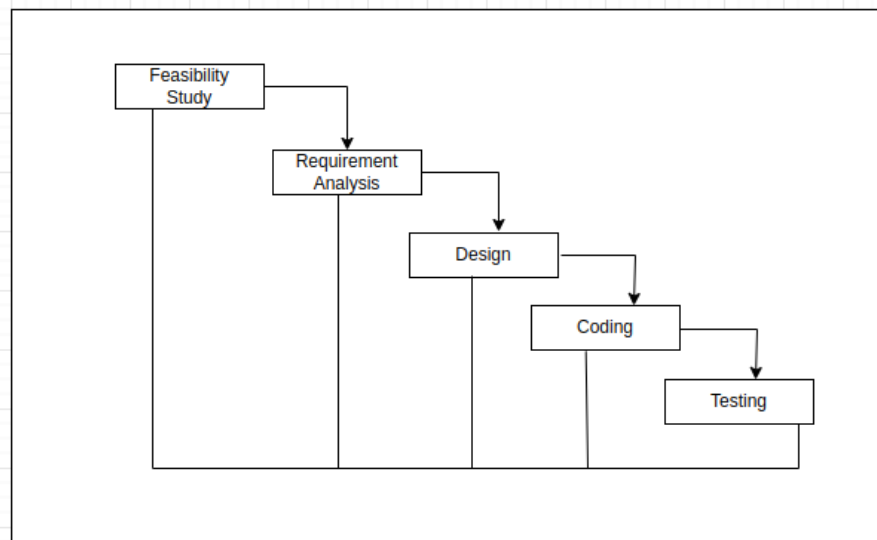


Figure 1: Waterfall Model for Grocery Delivery System

Gantt Chart

Title	Start	Finish	Duration	2/15/2025		2/28/2025		3/10/2025				4/18/2025					5/18/2025	
				W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Requirement Analysis	2/15/2025	2/28/2025	14 days															
System Design	2/28/2025	3/10/2025	10 days															
Implementation	3/10/2025	4/18/2025	35 days															
Testing	4/18/2025	5/18/2025	30 days															
Deployment	5/18/2025	5/30/2025	12 days															

Figure 2: Gantt Chart for Grocery Delivery System

1.6 Report Organization

The title of this project is “HARIYOBABAZAR an E-Commerce Web Application”. This report is organized into five main chapters, each of which documents the key aspects of the system’s design, development, and implementation.

1. **Introduction:** This chapter provides a general overview of the project. It includes the background of e-commerce platforms, the problem being addressed, the specific objectives of the system, the scope for both customers and sellers, and the limitations encountered during development. It also explains the development methodology followed throughout the project and outlines the structure of the report
2. **Background Study and Literature Review:** This chapter presents the background knowledge required to understand the project, including the MERN stack technologies used. It also includes a review of similar existing systems, previously conducted research, and relevant literature that supports the design and approach of the HARIYOBABAZAR platform.
3. **System Analysis and Design:** This chapter covers the detailed analysis and design of the system. It includes the functional and non-functional requirements, feasibility study, use case diagrams, data flow diagrams (DFD), entity relationship diagrams (ERD), and system architecture. It also explains how the data is modeled and structured, along with the design of UI/UX components and database schema
4. **Implementation and Testing:** This chapter describes the actual implementation of the system, including the technologies and tools used (React, Tailwind CSS, Node.js, Express, MongoDB, Cloudinary, Stripe, etc.). It provides details about each module, such as authentication, product management, and order handling. It also presents test cases for unit testing and system testing to validate the system's performance and reliability.
5. **Conclusion and Future Recommendations:** This chapter summarizes the key takeaways and lessons learned during the development of HARIYOBABAZAR. It reflects on how the objectives were achieved and discusses the practical impact of the project. It also outlines possible future enhancements such as real-time payment processing, delivery tracking integration, advanced filtering, and admin panel development.

CHAPTER 2: LITERATURE REVIEW

2.1 Background Study

This section of the report focuses on two essential areas: requirement identification and the study of existing systems.

Requirement Identification:

To design a grocery-focused e-commerce platform like HARIYOBAZZAR, it was important to first identify what customers and local grocery sellers need. Requirements were identified through observation of local grocery trends, user behavior on existing platforms, and understanding common challenges faced by both buyers and sellers. The core needs included secure login systems, categorized product listings, easy cart management, order tracking, seller-side product and stock control, and secure payment handling. These requirements formed the blueprint of HARIYOBAZZAR.

Study of Existing Systems:

The study of similar grocery delivery platforms helped identify common features, user expectations, and areas for improvement. In Nepal, platforms like MeroKirana, Karyoo, and BhojDeals are actively used by customers for purchasing groceries and food online. Each platform offers a unique approach to solving the grocery delivery challenge in the digital age. By analyzing them, HARIYOBAZZAR was designed to match industry standards while also targeting the local user experience with better control for sellers.

2.2 Literature Review

This literature review focuses on three existing grocery/food delivery platforms in Nepal—MeroKirana, Karyoo, and BhojDeals—that are relevant to the concept and implementation of HARIYOBAZZAR.

MeroKirana

MeroKirana is a popular online grocery delivery platform that offers a wide range of household products, fresh produce, beverages, and personal care items. The platform allows customers to order directly from its web interface and delivers to select areas of Kathmandu Valley. It provides users with features like real-time stock visibility, scheduled delivery slots, and digital payment options. However, MeroKirana operates with a centralized seller model—only their in-house team manages stock and delivery. In contrast,

HARIYOBAZZAR empowers multiple local sellers to manage their own product listings, stock, and orders under a shared e-commerce interface.

Karyoo

Karyoo is another growing Nepali e-commerce platform that focuses on delivering daily essentials, groceries, and lifestyle products. It supports both Android and web platforms and focuses on user convenience, offering quick delivery and multiple payment options. While it offers good customer features, the seller-side control and multi-seller support are limited. HARIYOBAZZAR improves on this by giving each seller their own dashboard where they can manage orders, add products, monitor stock levels, and interact with users directly.

BhojDeals

BhojDeals is primarily known as a food delivery platform, offering meals from local restaurants, but it has also expanded into grocery delivery. It is location-based and works closely with food vendors and retailers. While BhojDeals focuses on delivery logistics and aggregation, its backend tools for vendors are limited. HARIYOBAZZAR, on the other hand, combines grocery product delivery with full inventory and product control for sellers, making it a more flexible and scalable solution for local shop owners who want their own digital shelf space.

Summary of Review

While MeroKirana, Karyoo, and BhojDeals provide digital solutions for grocery and food delivery, they often operate in closed systems or with limited seller access. HARIYOBAZZAR introduces a shared multi-vendor model where sellers have full control over their inventory, products, and orders, while customers enjoy a smooth, categorized shopping experience with cart, checkout, and review systems.

This positions HARIYOBAZZAR as a locally focused, seller-friendly alternative that combines the convenience of online grocery shopping with the flexibility of independent store management.

Chapter 3: System Analysis and Design

3.1 System Analysis

To build HARIYOBAZZAR, we conducted a detailed system analysis by collecting data from both primary sources (interviews with local grocery sellers and online shoppers) and secondary sources (study of existing grocery e-commerce systems). Based on this information, the structural approach was chosen, as it best represents the system's workflow and data flow using diagrams like DFD and ERD.

The system follows a modular architecture, where each feature is separated into frontend and backend components. This improves maintainability, performance, and security.

3.1.1 Requirement Analysis

i. Functional Requirement

The following functional requirements were identified and implemented in the HARIYOBAZZAR system:

User Registration and Login: Users and sellers must be able to register and log in securely using their email (Gmail). JWT tokens and cookies are used to maintain login sessions.

User Profile Management: Users can update personal details such as name, address, and phone number from their profile page. Sellers can also edit store information and login credentials.

Product Browsing and Search: Customers should be able to browse products by categories, view product details, and search for grocery items easily through a search bar.

Cart Management: Users can add products to the cart, update quantities, remove items, and proceed to checkout.

Order Placement and Tracking: Customers can place orders after entering address and payment info. After placing an order, they can view current and past order statuses from the dashboard.

Seller Dashboard for Product Management:

Sellers have a dedicated dashboard to: Add new grocery products with name, image, description, price, stock, and category. Edit or delete existing product listings. Monitor stock levels and update them as needed.

Order Management for Sellers: Sellers can view incoming orders, update delivery status (e.g., pending, delivered), and manage customer information.

Customer Reviews and Ratings: After receiving an order, users can rate products and leave reviews.

Contact/Support System: Users can send messages through the contact form, which are received and managed by sellers from their dashboard.

Payment Integration (Test Mode): Users can simulate online payment using Stripe in test mode. The system handles payment tokens and confirms successful payment on the backend

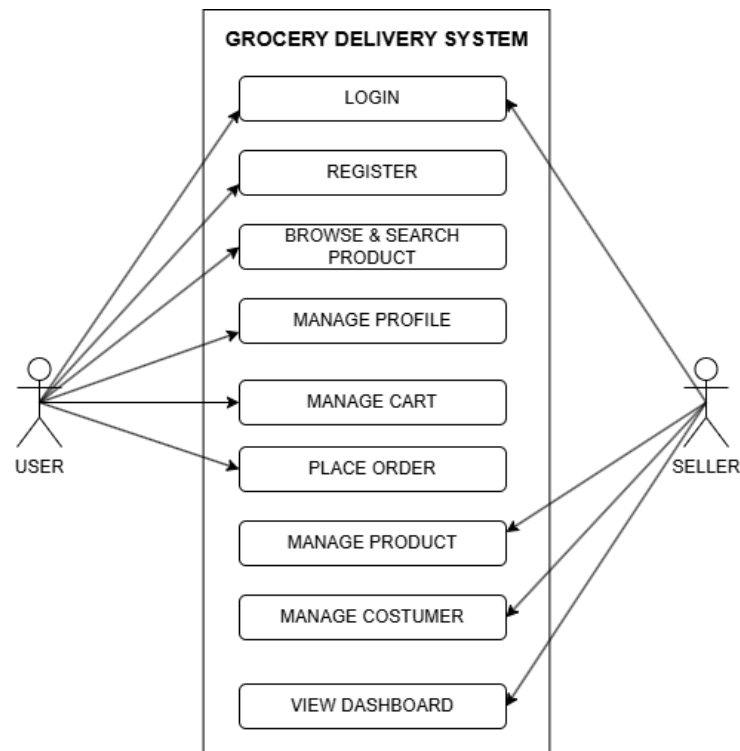


Figure 3: Use Case Diagram Of Grocery Delivery System

The figure below represents the Use Case Diagram of HARIYOBABAZAR – Grocery Delivery System. The system is designed with two primary user roles: Customer and Seller.

Customers can register/login using their Gmail accounts. Once authenticated, they can browse available grocery products categorized by type, view product details, add items to their cart, and place orders. Customers can also manage their profile, delivery addresses, and view their order history. Additionally, they can submit reviews and ratings for purchased products and contact support through a message form.

Sellers can register/login through a dedicated seller portal. Once logged in, sellers can access a dashboard that allows them to add new products, edit existing product details, manage stock availability, and upload images through Cloudinary. Sellers can view all incoming orders placed by users, update the order status (e.g., confirmed, out for delivery, delivered), and manage interactions through the message system.

The use case highlights the interactions between users and the system, where customers initiate product search and order requests, and sellers respond by fulfilling and managing those orders. This model allows for efficient grocery delivery by decentralizing product control to individual sellers, while maintaining a unified platform for users to shop easily.

ii. Non-Functional Requirements

Responsiveness:

The system must work smoothly on mobile phones, tablets, and desktop browsers.

Security:

Secure authentication via JWT and cookies, input validation, and restricted access to user/seller-only areas.

Usability:

The interface must be simple, clean, and intuitive, even for first-time users.

Performance:

The system must load quickly with minimal delays in page transitions, search, and cart updates.

Scalability:

The architecture must allow easy future upgrades, such as adding a mobile app or admin dashboard.

3.1.2 Feasibility Study

1. Technical Feasibility

The overall design and architecture of the project are technically straightforward. Both the frontend and backend components utilize widely adopted technologies like React, Node.js, Express, and MongoDB, which are readily available and well-documented. The UI design is kept clean and minimalistic, and all necessary tools, frameworks, and cloud services (such as hosting platforms and image storage solutions like Cloudinary) are easily accessible in the market. Therefore, the project is technically feasible with no significant limitations.

2. Operational Feasibility

The system is designed to address real-world challenges by enabling users to easily search, book, and interact with service providers. With a simple and user-friendly interface, the platform reduces the time and effort required to find reliable help. The application includes essential features like registration, service listing, booking, and chat, all of which enhance usability. The project will evolve over time through continuous feedback from users, making it highly operational in a real-world environment.

3. Economic Feasibility

The initial cost of development is reasonable and manageable. Most of the technologies used in the development process are open-source, significantly lowering expenses. For cloud services such as hosting, database management, and media storage, affordable plans are available that fit within a modest budget. As the system gains users and grows, the cost can be scaled proportionally. Thus, from a financial perspective, the project is economically viable and sustainable in the long run.

3.1.3 Data Modeling (ER-Diagram)

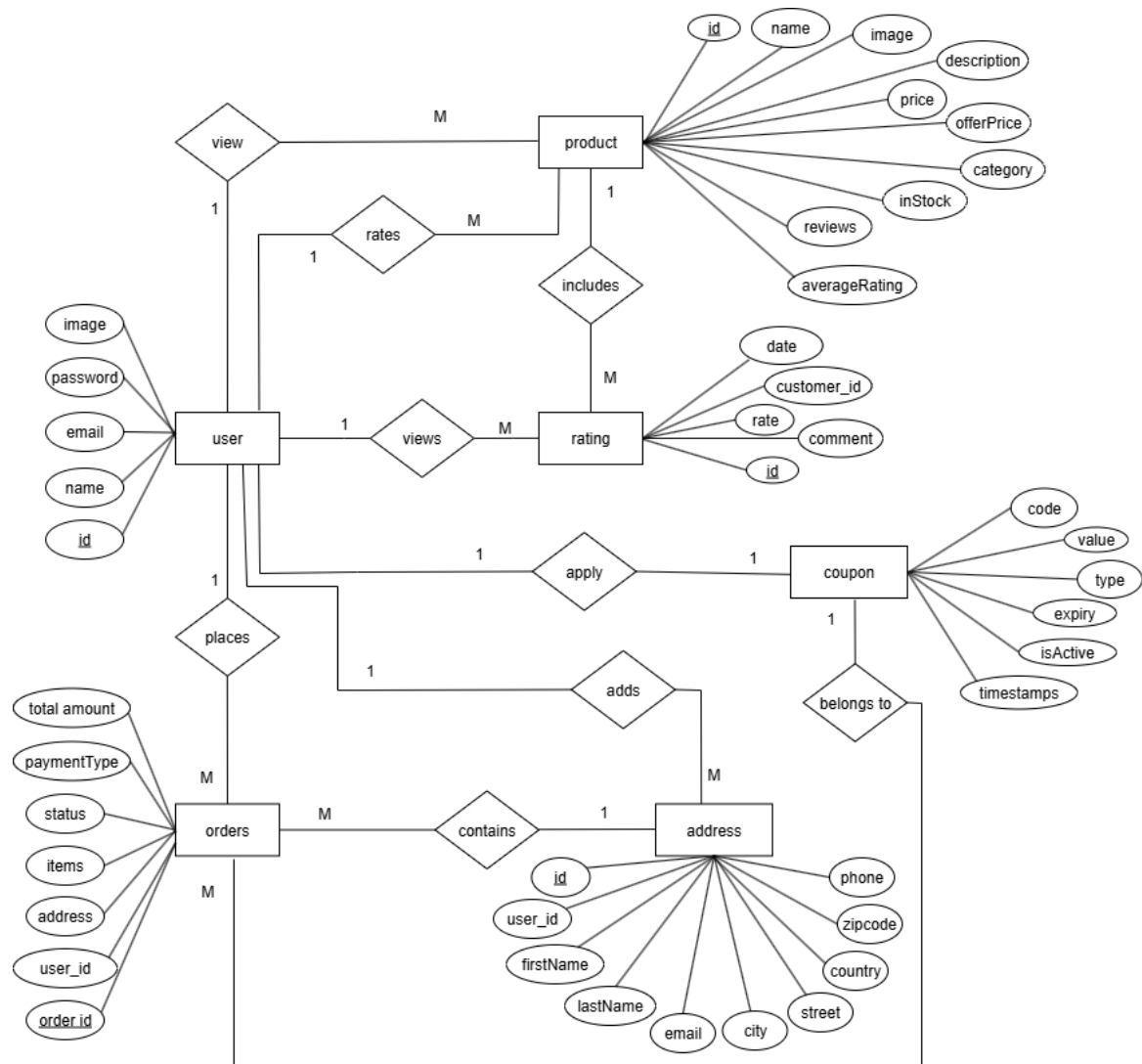


Figure 4: ER-Diagram of Grocery Delivery System

The above figure presents the Entity-Relationship (ER) Diagram of the Inventory and Order Management System. This diagram visualizes the core components of the system and how they interact, using standard ERD notations:

- Rectangles represent entities such as User, Order, Product (Inventory), Seller/Admin, Address, and Review.
- Diamonds depict relationships between entities like Places, Manages, Contains, Has, Ships To, and Writes.
- Ovals denote attributes of entities, such as name, email, amount, status, etc.
- Lines with cardinality (1:N, N:M) define how many instances of one entity relate to another

3.1.4 Process Modeling DFD

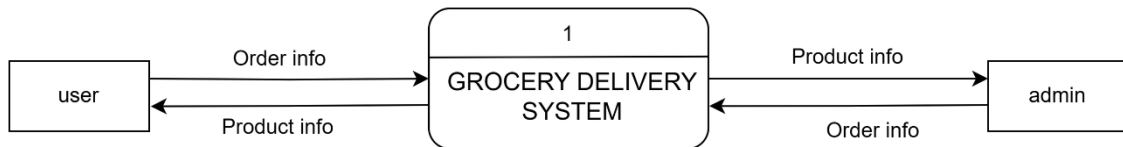


Figure 5: Data Flow Diagram of Grocery Delivery System

The above figure illustrates the Data Flow Diagram (DFD) of the HARIYOBAZZAR System, outlining the flow of information between the primary actors—Customer (User) and Seller (Admin)—and the central system.

This DFD highlights the key processes and interactions within the system, showing how users and sellers exchange data through the HARIYOBAZZAR platform.

1. Customer (User) Interaction

Customers interact with the system through the following data flows:

Inputs to the System:

- User Details: Registration, login credentials, and profile information.
- Place Orders: Customers browse products and place purchase orders.
- Search Queries: Users search for products using keywords or categories.
- Reviews: Users leave feedback or ratings for purchased products

Outputs from the System:

- Product Info: Product names, prices, availability, and categories.
- Order Status: Real-time order tracking and delivery status updates.
- Authentication Confirmation: Login validation and session management.

2. Seller (Admin) Interaction

Sellers (Admins) use the system to manage inventory and track customer orders:

Inputs to the System:

- Add/Update Products: Sellers can upload new products or edit existing listings.
- Update Order Status: Manage the processing, shipping, and delivery of orders.

Outputs from the System:

- Customer List: Access to basic user data for order fulfillment.
- New Orders: Notification and details of incoming customer orders.
- Inventory Details: Overview of stock levels and product availability.
- Dashboard Stats: Visual and statistical summaries of system activity (sales, customers, etc.)

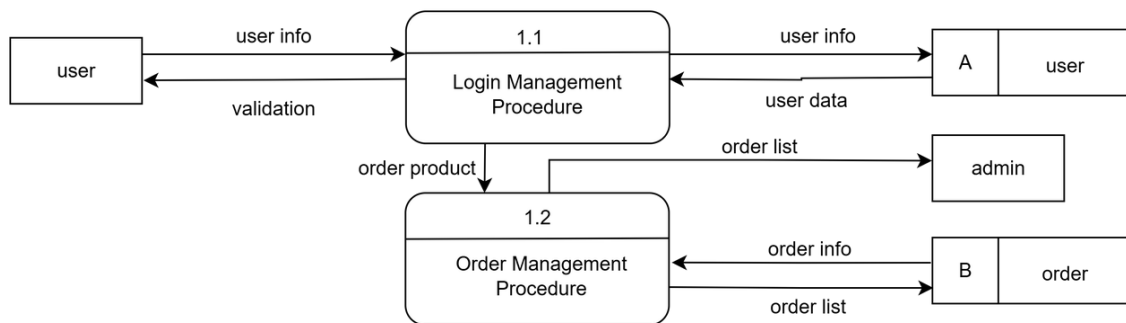


Figure 6: Level one Context Diagram of Grocery Delivery System

The above figure represents the module-level data flow architecture of the HARIYOBZZAR System, which demonstrates the interaction between the main actors User and Seller (Admin) with the system and its backend database.

The architecture is divided into three layers:

- Actors
- System Modules
- Database Layer

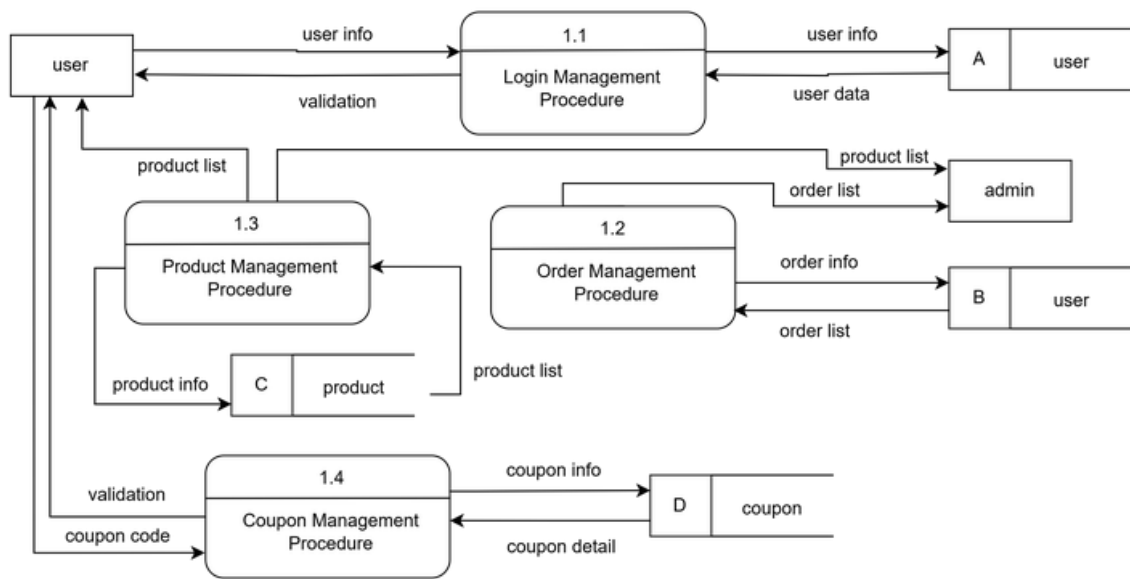


Figure 7: Level two Context Diagram of Grocery Delivery System

3.2 System Design

3.2.1 Architectural Design

We have used MERN stack thus the architecture of the system is designed samillary as:

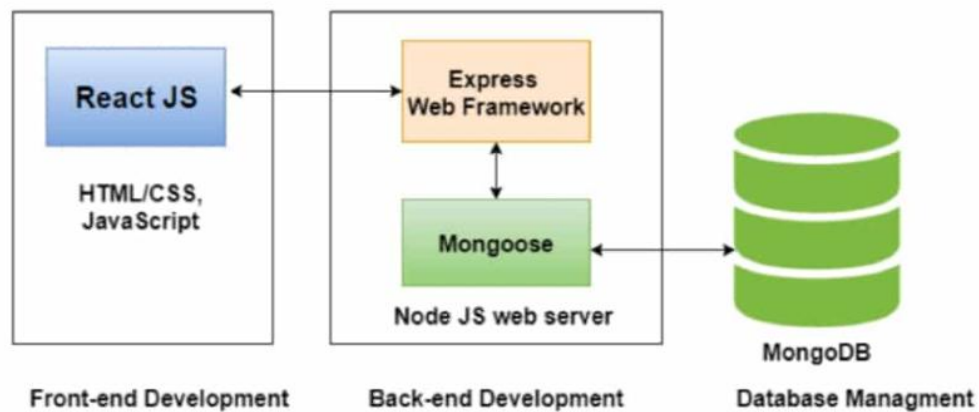


Figure 8: Architectural Design of Grocery Delivery System

React JS is frontend and Nodejs is backend. The node js interacts with MongoDB ad DBMS. This is a rest api architecture where express and Nodejs is library like mongoose interacts to MongoDB for fetching and manipulating of data as well as creation of schema

Chapter 4: IMPLEMENTATION AND TESTING

4.1 Implementation

Various tools and technologies were used during the development of the HariyoBazaar luxury grocery e-commerce platform. Below is a summary of the tools and their purposes.

4.1.1 Tools Used

Table 1: Tools Used for Project

Tools	Purpose
Visual Studio Code	Code editing and development
React.js	Frontend development
Tailwind CSS	Frontend styling
JavaScript (ES6+)	Frontend logic and interactivity
Node.js	Backend runtime environment
Express.js	Backend web framework
MongoDB	Database for storing application data
Mongoose	MongoDB object modeling
Cloudinary	Image storage and management
Multer	File upload handling
JWT	Authentication and authorization
Draw.io	Creating diagrams and system architecture figures

4.1.2 Implementation details of modules

1. Visual Studio Code:

Used as the primary code editor for both frontend and backend development, offering features like syntax highlighting, extensions, integrated terminal, and Git integration.

2. React.js

Utilized for building the user interface, enabling component-based architecture, fast rendering, and a responsive, interactive user experience.

3. Tailwind CSS

Used for efficient and maintainable styling, allowing rapid prototyping and a consistent design system across the application.

4. JavaScript (ES6+):

The main programming language for frontend logic, enabling dynamic content, form validation, and API integration.

5. Node.js:

Provides the runtime environment for executing backend JavaScript code, handling server-side logic and API endpoints.

6. Express.js:

A lightweight web framework for Node.js, used to build RESTful APIs, handle routing, middleware, and server-side logic.

7. MongoDB:

A NoSQL database used to store user data, product information, orders, addresses, and more, supporting flexible and scalable data models.

8. Mongoose:

An ODM (Object Data Modeling) library for MongoDB, used to define schemas and interact with the database in an organized way.

9. Cloudinary:

Used for secure and scalable image storage and management, especially for product images.

10. Multer:

Middleware for handling file uploads from the client to the server.

11. JWT (JSON Web Token):

Used for secure user authentication and authorization across the application.

12. Draw.io:

Used to create system architecture diagrams, flowcharts, and other visual documentation

4.2 Testing

Testing ensures that the HariyoBazzar platform meets user requirements and system specifications, and helps identify and fix defects. Both unit and integration testing were performed.

4.2.1 Test Cases for Unit Testing

Table 2: Test result for Login Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	Login form with email and password fields.	User is logged in and redirected to the homepage.	User successfully logged in and redirected.	PASS
2	Incorrect or empty credentials.	Error message is displayed.	Error message shown for invalid credentials.	PASS

Table 3: Test result for Register Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	Registration form with user details.	User receives a registration success message.	User notified of successful registration.	PASS
2	Duplicate email or invalid input.	Error message is displayed.	Error message shown for duplicate/invalid input.	PASS

Table 4: Test result for Product Browsing & Details Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	User browses products by category or search.	Products are displayed as per filter/search.	Products displayed correctly.	PASS
2	User clicks on a product.	Product details page is shown.	Product details loaded successfully.	PASS

Table 5: Test result for Cart & Checkout Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	Add product to cart.	Product appears in cart.	Product added to cart.	PASS
2	Proceed to checkout with valid details.	Order is placed and confirmation shown.	Order placed and confirmation received.	PASS
3	Checkout with missing/invalid details.	Error message is displayed.	Error message shown for invalid input.	PASS

Address Management Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	Add new address.	Address is saved and listed.	Address added successfully.	PASS
2	Add address with missing fields.	Error message is displayed.	Error message shown for missing fields.	PASS

Seller Product Management Module

Table 6: Test result for Seller Product Management Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	Seller adds a new product.	Product is added and visible in product list.	Product added successfully.	PASS
2	Seller edits product details.	Product details updated.	Product updated successfully.	PASS
3	Seller tries to add product with missing info.	Error message is displayed.	Error message shown for missing info.	PASS

Table 7: Test result for Order Management Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	User views order history.	List of past orders is displayed.	Order history displayed.	PASS
2	Seller views/manage orders.	Seller can see and update order status.	Seller managed orders successfully.	PASS

Table 8: Test result for Order Management Module

S.N.	Description	Expected Result	Actual Result	Test Result
1	User updates profile info.	Profile updated successfully.	Profile updated.	PASS
2	Invalid profile update (e.g., blank fields).	Error message is displayed.	Error message shown for invalid input.	PASS

Chapter 5: RESULT CONCLUSION AND REMAINING WORK

5.1 Result

By working on the HariyoBazaar project, I gained extensive experience in developing a web application using modern tools and technologies. I learned to design and implement both frontend and backend systems, manage databases, and integrate third-party services such as Cloudinary for image storage. The project also enhanced my skills in documentation, version control, and collaborative development. Presenting the project further improved my communication and confidence. Most importantly, I successfully developed a luxury-themed e-commerce platform that streamlines grocery shopping and seller management, providing a seamless and user-friendly experience for both customers and sellers.

5.2 Conclusion

HariyoBazaar, a luxury grocery e-commerce platform, was successfully developed using a modern web technology stack including React.js, Tailwind CSS, Node.js, Express.js, and MongoDB. The system follows a modular, scalable architecture and implements essential features such as user authentication, product browsing, cart management, order processing, and a dedicated seller dashboard. The platform improves the efficiency and accuracy of online grocery transactions, enhances user convenience, and demonstrates the importance of integrating advanced technology in retail management. The project has laid a strong foundation for a robust, scalable, and visually appealing e-commerce solution.

5.3 Remaining work

While the core functionalities have been implemented, several enhancements can further improve the system:

Payment Gateway Integration: Implement secure online payment options for seamless checkout.

Advanced Accounting Features: Add modules for accounting, including voucher generation and financial reporting.

Product Types, Brands, and Attributes: Expand product catalog management to include detailed product types, brands, and attribute filtering.

PDF Report Generation: Enable users and sellers to generate and download order and sales reports in PDF format.

Wishlist and Payment Methods: Add wishlist functionality and allow users to save multiple payment methods for convenience.

Real-Time Order Tracking: Implement live order status updates and delivery tracking for users.

Reference

- [1] <https://www.sciencedirect.com/science/article/abs/pii/S1366554521002295>
- [2] <https://ieeexplore.ieee.org/abstract/document/8291927/>
- [3] <https://www.sciencedirect.com/science/article/pii/S1110016820303185>
- [4] <https://link.springer.com/article/10.1007/s00500-016-2406-5>